ORM У NODE.JS

TypeORM deep dive та альтернативи: entities, міграції, relations, N+1, QueryBuilder, транзакції, Kysely

Заняття №13

Node.js Pro

Лектор
Кирило Орлов

СТЕК НА СЬОГОДНІ: PG · TYPEORM · POSTGRES 16 · KYSELY

Перша половина — TypeORM core	Друга половина — межі й альтернативи
крок 1 · голий pg: біль руками	крок 5 · N+1: 11 запитів → 1
крок 2 · entity + Repository	крок 6 · QueryBuilder + soft delete
крок 3 · synchronize vs міграції	кроки 7-8 · транзакції і Kysely
крок 4 · relations: M:N із даними	ORM економить write-path. Платиш на read-path

НАСКРІЗНА НИТКА
logging: ['query'] увімкнено всю лекцію: усе, що ORM робить за спиною, видно тільки в лозі SQL. Кожен крок — окремий скрипт проти живого Postgres на 21300.

ПЛАН ЗАНЯТТЯ



- Голий pg: чому мапінг руками не масштабується
- Entity + Repository: помилка їде з рантайму в tsc
- synchronize проти міграцій: хто веде схему
- Relations: M:N із даними, RESTRICT проти CASCADE
- N+1: детект у лозі SQL і три фікси
- QueryBuilder, soft delete, транзакції
- Kysely + матриця альтернатив · ДЗ по курсовому

СЬОГОДНІ ПОБАЧИТЕ НА ВЛАСНІ ОЧІ

01. synchronize

«деплой №2» мовчки DROPнув колонку разом із даними

02. N+1

невинний find() у циклі — 11 запитів замість одного

03. JOIN

той самий граф одним запитом: relations: { items }

04. soft delete

DELETE, який насправді UPDATE deletedAt = now()

05. rollback

замовлення на 99 штук відкотилось — склад не змінився

06. Kysely

той самий звіт без ORM: числа збігаються один в один

ПРОБЛЕМА: СХЕМА ЖИВЕ У ДВОХ СВІТАХ

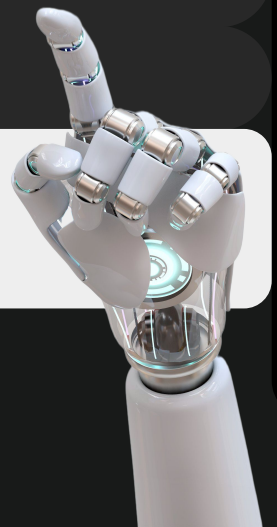
у коді — об'єкти й типи, у БД — таблиці й рядки

мапінг row → об'єкт руками в кожному запиті

назви колонок — рядки: tsc їх не перевіряє

одруківка в SQL живе до рантайму — і до проду

Питання лекції — не «чи ORM хороший». Питання: що він робить ЗА НАС, що — ЗА СПИНОЮ, і де його межа.



```
СВІТ КОДУ

interface User {
  id: number
  fullName: string
}

  ↕ мапінг руками

СВІТ БД

raw_users
  id        serial
  full_name varchar

_____________________

'SELECT full_nmae…'
tsc     → мовчить
рантайм → ✗ column
  does not exist
```

КРОК 1 · ГОЛИЙ pg: ПРАЦЮЄ, ПОКИ МАЛЕНЬКЕ

```
import { Client } from 'pg';

interface User {
  id: number;
  email: string;
  fullName: string;                  // у БД колонка full_name
}

const res = await client.query(
  'SELECT id, email, full_name FROM raw_users ORDER BY id');
const users: User[] = res.rows.map((r) => ({
  id: r.id,
  email: r.email,
  fullName: r.full_name,             // міст між світами — руками
}));
console.log('OK, змапили руками:', users);  // [Ada, Linus]

// типова прод-помилка: одруківка в НАЗВІ КОЛОНКИ
await client.query('SELECT id, email, full_nmae FROM raw_users');
// ✗ column "full_nmae" does not exist ← вибухнуло в РАНТАЙМІ
```

Стартова точка — драйвер pg, як на L#12

1 · interface User — БД про нього не знає

2 · мапінг full_name → fullName у кожному запиті

3 · full_nmae: tsc компілює мовчки

вивід: ✗ column "full_nmae" does not exist

урок: 40 запитів по проєкту = 40 мапінгів руками

«ПЕРЕКЛАДАЧ-СИНХРОНІСТ»: ORM ЯК АНАЛОГІЯ

ЧАСТИНА ORM

- @Entity + @Column — мапінг
- repo.find({ where }) — запит
- logging: ['query'] — лог SQL
- QueryBuilder — свій SELECT
- ds.query(...) — чистий SQL

АНАЛОГІЯ (ПЕРЕКЛАДАЧ)

- словник термінів двох мов
- кажеш думку — фразу будує він
- стенограма: що він РЕАЛЬНО сказав
- диктуєш майже дослівно
- говориш мовою оригіналу сам

Де аналогія ламається: платиш за КОЖНУ фразу — «про кожне замовлення окремо» стане N+1 дзвінками, він не спитає. І контракт (звіт із GROUP BY) він не складе — там говориш SQL сам.



КРОК 2 · ENTITY + REPOSITORY: ЩО ЗНИКЛО

мапінг робить ORM — за метаданими декораторів

властивість = поле класу: tsc бачить назви

where: { fullNmae } тепер НЕ компілюється

save() = INSERT або UPDATE; SQL — у лозі

збірка тільки tsc: esbuild/tsx метадані не емітять

synchronize: true — поки що; чому не можна — далі

```
ЩО ВІДБУВАЄТЬСЯ ЗА СПИНОЮ

@Entity() class User {
  @Column() email
  @Column() fullName
}
     | метадані типів
     ▼
CREATE TABLE "users_step2"
INSERT … RETURNING "id"
SELECT … WHERE "isActive"
     | рядки з БД
     ▼
repo.find() → User[]
     типи, без map
```



```
// Крок 2: той самий користувач — тепер entity + Repository
import 'reflect-metadata';
import { Entity, PrimaryGeneratedColumn, Column, Index } from 'typeorm';

@Entity('users_step2')
class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Index({ unique: true })
  @Column({ type: 'varchar', length: 255 })
  email: string;

  @Column({ type: 'varchar', length: 100 })
  fullName: string;        // колонка "fullName" — без snake_case
}

const users = await repo.find({ where: { isActive: true } });
// → User[] без map і без as; SQL цього find — у лозі
// find({ where: { fullNmae } }) — помилка КОМПІЛЯЦІЇ, не рантайму
```



```
// Крок 3а: чому synchronize: false — єдина опція для спільних БД

// «Деплой №1»: таблиця з full_name, у ній живі дані
@Entity('users_danger') class UserV1 {
  @Column({ name: 'full_name' }) fullName: string;
}

// «Деплой №2»: типовий рефакторинг — розбили на first/last
@Entity('users_danger') class UserV2 {
  @Column({ name: 'first_name', nullable: true })
  firstName: string | null;
  @Column({ name: 'last_name', nullable: true })
  lastName: string | null;
}

// synchronize бачить: full_name зник, first/last з'явились →
//   ALTER TABLE … DROP COLUMN "full_name"  ← мовчки, РАЗОМ З ДАНИМИ
// лог схеми: columns dropped: full_name

// До деплою №2: 'Ada Lovelace', 'Linus Torvalds'
// Після:        first_name = null, last_name = null
```



КРОК 3Б · МІГРАЦІЇ ЗАМІСТЬ SYNCHRONIZE

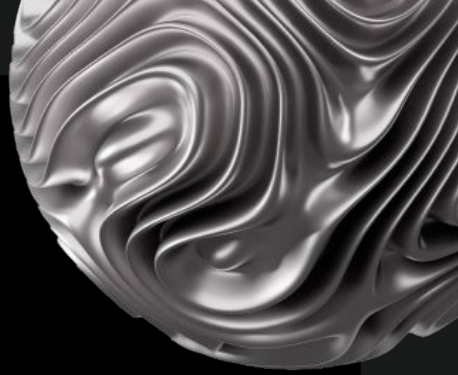
migration:generate пише обидві половини: up() — CREATE, down() — DROP

run → CREATE TABLE users_migr + запис у "migrations"; show → [X] Init1785784320098

revert → down(): DROP TABLE; show → [ ] знову pending — цикл повторюваний



згенероване ЧИТАТИ перед комітом: генератор не знає про дані — NOT NULL на заповненій таблиці впаде, перейменування бачить як DROP + ADD. CLI працює зі скомпільованим DataSource (-d dist/03-data-source.js), і згенерований .ts теж треба зібрати



```
// Крок 4: M:N Order ⇔ Product — НЕ @ManyToMany: на зв'язку живуть ДАНІ
@Entity('order_items')
export class OrderItem {
  // видалили замовлення → позиції без сенсу → CASCADE
  @ManyToOne(() => Order, (o) => o.items, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'orderId' })
  order: Order;

  // товар із продажами видаляти заборонено → RESTRICT
  @ManyToOne(() => Product, { onDelete: 'RESTRICT' })
  @JoinColumn({ name: 'productId' })
  product: Product;

  @Column({ type: 'int' }) quantity: number;
  @Column({ type: 'int' }) priceAtPurchaseCents: number; // на МОМЕНТ
}
// Order:   @OneToMany(() => OrderItem, i => i.order, { cascade: true })
// Product: priceCents int (гроші = копійки) + @DeleteDateColumn()
```

КРОК 4 НАЖИВО: ДАНІ ЗАХИЩАЄ БД, НЕ ORM

RESTRICT

delete(товар із продажами) → violates foreign key constraint — Postgres не дав

CASCADE

remove(order) → позицій було 20, стало 18: діти пішли за батьком на рівні БД

cascade: true

опція save(): зберегти order разом з items одним викликом — про запис, не видалення

Чому це важливо саме зараз

onDelete живе у СХЕМІ БД: спрацює і повз ORM — із psql, з міграції, з іншого сервісу. Класика співбесіди: чим cascade: true відрізняється від onDelete: 'CASCADE'? Перше — про збереження графа, друге — про видалення.

КРОК 5 · N+1: ЛІЧИЛЬНИК, 10 ЗАМОВЛЕНЬ

Як читаємо	Запитів	Коли це ок
find() + find у циклі	11 = 1 + 10	ніколи: росте разом із N
relations: { items: true }	1 (LEFT JOIN)	дефолт для графа об'єктів
leftJoinAndSelect(...)	1	коли треба WHERE по зв'язку
relationLoadStrategy: 'query'	3 — константа	JOIN роздуває рядки (1 × 200)
10 000 замовлень	10 001 у наївного	фікси лишаються 1–3

ДЕ N+1 ХОВАЄТЬСЯ У ЖИВОМУ КОДІ

01. цикл: find() списку + запит на елемент у тілі — наш крок 5



02. lazy-поля: await order.items виглядає як поле, а є запитом



03. серіалізація: маппер тягне relation поза очима

04. GraphQL-резолвери: N+1 на кожному рівні — ліки DataLoader, L#17

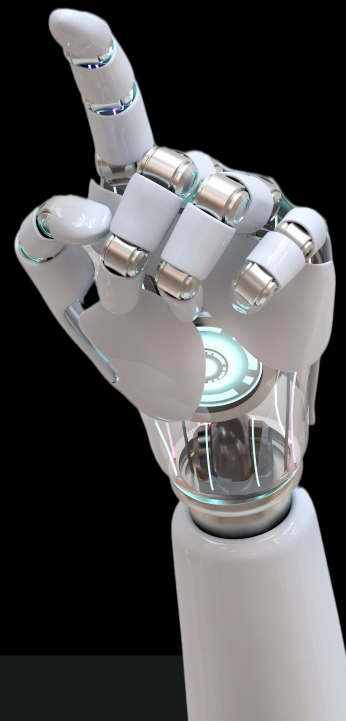


05. детектор один: лог SQL або лічильник запитів у Logger

КАРТА ІНСТРУМЕНТІВ: ВІД find() ДО RAW SQL

```
find / findOne / save      граф об'єктів, CRUD      ~70% запитів
     ↓
relations · order · take   усе ще типізований граф
     ↓
  ── ТУТ ЗАКІНЧУЄТЬСЯ ГРАФ ОБ'ЄКТІВ ──────────────────
     ↓
createQueryBuilder()       SELECT довільної форми:
  .innerJoin()             агрегати, GROUP BY, підзапити
  .groupBy()
  .getRawMany<Row>()       «рядки звіту», НЕ entity
     ↓
ds.query('SELECT …', [])   вікна, CTE — усе, що вмієш з L#12

нижче по драбині — більше контролю, менше «за спиною»
```



SOFT DELETE: ЩО РОБИТЬ КОЖЕН ВИКЛИК

Виклик	SQL за спиною	Видно товарів
softRemove(hub)	UPDATE … SET "deletedAt" = now()	було 5
find()	сам додає WHERE "deletedAt" IS NULL	4
find({ withDeleted: true })	той самий SELECT без фільтра	5
restore(id)	UPDATE … SET "deletedAt" = NULL	знову 5
delete(id)	справжній DELETE — повз soft delete	RESTRICT ще діє

```
// Крок 6: звіт, який find() не вміє — GROUP BY + SUM по join-таблиці
const report = await ds.getRepository(Product)
  .createQueryBuilder('p')
  .innerJoin('order_items', 'oi', 'oi."productId" = p.id')
  .select('p.name', 'name')
  .addSelect('SUM(oi.quantity)', 'units')
  .addSelect('SUM(oi.quantity * oi."priceAtPurchaseCents")',
             'revenueCents')
  .groupBy('p.id')
  .orderBy('"revenueCents"', 'DESC')
  .getRawMany<RevenueRow>(); // units: string — bigint > number

// Custom repository: доменний метод замість копіпасти по сервісах
const productRepo = ds.getRepository(Product).extend({
  findLowStock(t: number) {
    return this.createQueryBuilder('p')
      .where('p.stock < :t', { t }).orderBy('p.stock', 'ASC').getMany();
  },
});

// Вивід: Ноутбук units=6 revenueCents=25200000 → 252 000 грн
```



КРОК 7 · ТРАНЗАКЦІЇ: ВСЕ АБО НІЧОГО

списати склад + створити замовлення — атомарно, або фантомний продаж

QueryRunner = одне з'єднання з пулу під твоїм контролем

всередині — ТІЛЬКИ runner.manager: repo піде іншим з'єднанням

finally { release() } — інакше з'єднання витече з пулу

lock: pessimistic_write = SELECT ... FOR UPDATE — деталі на L#14

вивід: склад 5 → 3; спроба 99 → rollback: склад 3, замовлень 0

```
// Крок 7: commit або rollback — без середини
const runner = shopDataSource.createQueryRunner();
await runner.connect();
await runner.startTransaction();
try {
  const product = await runner.manager.findOneOrFail(Product, {
    where: { name },
    lock: { mode: 'pessimistic_write' },  // SELECT … FOR UPDATE
  });
  if (product.stock < qty) throw new Error('Недостатньо на складі');
  product.stock -= qty;
  await runner.manager.save(product);
  // … Order + OrderItem — тим самим runner.manager …
  await runner.commitTransaction();
} catch (e) {
  await runner.rollbackTransaction();     // склад НЕ змінився
  throw e;
} finally {
  await runner.release();                 // інакше з'єднання витече
}
// коротший шлях: await ds.transaction(async (m) => { … })
```



АЛЬТЕРНАТИВИ: ХТО ВОЛОДІЄ СХЕМОЮ І SQL

ІНСТРУМЕНТ	МОДЕЛЬ
• Prisma • Drizzle • Kysely — крок 8 • raw pg — L#12 • критерій вибору	• свій DSL schema.prisma + generate, SQL схований найглибше • схема в TS, SQL-подібний білдер, легкий рантайм • типізований SQL-білдер без ORM: схема = interface • повний контроль, мапінг руками • де живе схема і хто пише SQL

```
// Крок 8: той САМИЙ звіт — на Kysely. Схема = звичайні interface
interface Database {
  products:    { id: string; name: string; priceCents: number };
  order_items: { orderId: string; productId: string;
                 quantity: number; priceAtPurchaseCents: number };
}

const report = await db         // db = new Kysely<Database>({ … })
  .selectFrom('products as p')
  .innerJoin('order_items as oi', 'oi.productId', 'p.id')
  .select((eb) => ['p.name',
    eb.fn.sum<string>('oi.quantity').as('units'),
    eb.fn.sum<string>(eb('oi.quantity', '*',
      eb.ref('oi.priceAtPurchaseCents'))).as('revenueCents')])
  .groupBy('p.id').orderBy('revenueCents', 'desc')
  .execute();   // 'oi.qty' — помилка КОМПІЛЯЦІЇ, не рантайму

// Числа 1:1 з кроком 6: Ноутбук 25200000, Монітор 5460000, …
// Немає прихованих завантажень → немає N+1 «з коробки». Але й
// save(граф), migration:generate і soft delete — теж немає.
```

ЩО СВІДОМО ЛИШАЄМО ЗА ДУЖКАМИ

TypeORM-глибини: subscribers / entity listeners, кеш запитів, lazy relations — у бонус-матеріалах; на ефірі чесно не встигнемо

сусідні теми: рівні ізоляції та локи (сьогодні — один рядок lock) → L#14 · пул, репліки, шардінг → L#15 · DataLoader по-справжньому → L#17

⚠ версії: у репо typeorm@0.3.31 (пін). З 19.05.2026 npm і typeorm ставить 1.x (зараз — 1.1.0, гілка 0.3 = dist-tag legacy). API всього, що сьогодні звучало, у 1.x той самий; пін — заради відтворюваності



ПІДСУМКИ + МІСТ ДО L#14

ENTITY

мапінг і типи безкоштовно; схему ведуть міграції з up() і down(), а не synchronize

МЕЖА

find() — граф об'єктів; звіти — QueryBuilder; далі — raw SQL чи Kysely. Це не зрада ORM — це його межа

ДАЛІ

L#14 — транзакції по-дорослому: рівні ізоляції, аномалії, FOR UPDATE / SKIP LOCKED

ДЗ #13

10 балів (PROJECT HEAVY) · дедлайн 10.09 · entities + relations + міграції для курсового зі схеми ДЗ #12 · fix N+1 із числами до/після · звіт через QueryBuilder

N+1

не видно в коді — видно в лозі SQL: 11 → 1 (JOIN) або 3 (query-стратегія)